

Esercizio B1 – Bit consecutivi – Punti ≥ 8

Ancora? Sì, ancora. Il problema si può risolvere tramite backtrack. La funzione ricorsiva, oltre al parametro i che identifica il prossimo bit da riempire, prende i parametri i_0 e i_1 che rappresentano il numero di bit 0 e 1 consecutivi che si possono scegliere, rispettivamente; e i parametri n_0, n_1 necessari per re-inizializzare questi valori quando si sceglie il bit opposto. All'inizio, $i = n$, $i_0 = n_0$ e $i_1 = n_1$.

Quando $i = 0$, tutti i bit sono stati scelti e si può stampare. Altrimenti, se $i_0 > 0$, si può ancora scegliere 0, si chiama ricorsivamente la funzione ($bRec()$) con un bit consecutivo in meno per i_0 e re-inizializzando i_1 a n_1 , perché abbiamo possiamo ricominciare con un'eventuale sequenza di 1. In modo simile, se $i_1 > 0$ è possibile chiamare ricorsivamente ($bRec()$) con un bit consecutivo in meno per i_1 e re-inizializzando i_0 a n_0 .

La complessità è $O(n \cdot 2^n)$; infatti, nel caso in cui $n_0 = n_1 = n$, verranno stampate tutte le 2^n stringhe di n bit.

```
binary(int n, int n0, int n1)
```

```
int[] S = new int[1..n]
bRec(S, n, n0, n1, n0, n1)
```

```
bRec(int[] S, int i, int i0, int i1, int n0, int n1)
```

```
if i == 0 then
  print(S)
if i0 > 0 then
  S[i] = 0
  bRec(S, i - 1, i0 - 1, n1, n0, n1)
if i1 > 0 then
  S[i] = 1
  bRec(S, i - 1, n0, i1 - 1, n0, n1)
```

Esercizio B2 – Discordville – Punti ≥ 11

Il problema, guarda un po', può essere risolto con programmazione dinamica in tempo lineare.

Sia $DP[i]$ la quantità massima di donazioni che possono essere ottenute considerando le sole prime i case. Ovviamente, la casa in posizione n può donare senza preclusioni, in quanto non ha nessuno dopo di lui. Quando si considera la posizione i -esima, è possibile seguire due scelte:

- se si prende la donazione della casa i -esima, poi si prende il massimo che si può ottenere con le prime $i - N[i] - 1$ case; se questo indice è minore di zero, si prende il valore $D[i]$ e basta;
- se non si prende la donazione della casa i -esima, si prende il massimo che si può ottenere con le prime $i - 1$ case.

Il valore massimo che stiamo cercando si trova in posizione $O[n]$.

Questo può essere espresso dalla seguente equazione ricorsiva:

$$DP[i] = \begin{cases} D[1] & i = 1 \\ \max\{DP[i-1], D[i]\} & i > 1 \wedge i - N[i] - 1 \leq 0 \\ \max\{DP[i-1], D[i] + DP[i - N[i] - 1]\} & i > 1 \wedge i - N[i] - 1 > 0 \end{cases}$$

Questo può essere tradotto nel seguente algoritmo basato su programmazione dinamica:

```
int discordville(int[] D, int[] N, int n)
```

```
int[] DP = new int[1..n]
DP[1] = D[1]
for i = 2 to n do
  int prec = if(i - N[i] - 1 > 0, DP[i - N[i] - 1], 0)
  DP[i] = max(DP[i-1], D[i] + prec)
return DP[n]
```

Esercizio B3 – Math ways – Punti ≥ 11

È possibile utilizzare la programmazione dinamica per risolvere il problema.

Sia $DP[i]$ il numero di modi per ottenere n a partire da i . Ovviamente, nel caso $i = n$, esiste un solo modo - non fare nulla. Altrimenti, $DP[i]$ è pari alla somma di $DP[i + 1]$, $DP[2i]$, $DP[i^2]$. Se $i > n$, abbiamo sfornato e ovviamente il numero di modi è pari a 0.

La formula ricorsiva per calcolare DP è quindi:

$$DP[i] = \begin{cases} 0 & i > n \\ 1 & i = n \\ DP[i + 1] + DP[2i] + DP[i^2] & \text{altrimenti} \end{cases}$$

Scriviamo il codice in maniera bottom-up:

```
int mathways(int n, int k)
int[] DP = new int[k...n]
DP[n] = 1
for i = n - 1 downto k do
    DP[i] = DP[i + 1]
    if 2 · i ≤ n then
        DP[i] = DP[i] + DP[2 · i]
    if i2 ≤ n then
        DP[i] = DP[i] + DP[i2]
return DP[k]
```

Il costo dell'algoritmo proposto è $\Theta(n)$.

Si noti che la richiesta che k sia maggiore di 3 è per non aver problemi nel distinguere $2k$ e k^2 quando $k = 2$. Ovviamente l'algoritmo proposto funziona anche nel caso $k = 1, 2$ se assumiamo che $2 \cdot 2$ e 2^2 siano due operazioni diverse. Altrimenti, è sufficiente inserire un caso particolare nella soluzione proposta.